



QA Workflows for the AI SDLC

"VALIDATION is a generation behind GENERATION. That gap is our job."

Sudhakar Kakunuri · QtpSudhakar · 21 years in QA · Founder, VibeTestQ

 AI-Native QA

 Real Tools

 Real Workflows

 Before I begin—



I've spent 21 years inside this industry. Trained 10,000+ engineers. Delivered 50+ keynotes.

So the least I owe this room...

is to tell you what we got wrong.

 Not AI. *Us*. The IT industry.

Mistake #1 — We Chose Disconnection



💬 Quick question: What tools does your team use for project management, code, and test cases? Are they from the same vendor?

What was offered

- ✓ Atlassian: Jira + Bitbucket (ready, integrated)
- ✓ Azure DevOps: PM + Repos + Test Plans
- ✓ GitLab: PM + Code + CI/CD

What we chose

- ✗ Jira for PM — but GitHub for code
- ✗ Azure DevOps for code — but Jira for PM
- ✗ TestRail separately. Confluence separately. Slack separately.

⚠️ We didn't inherit fragmentation. We chose it. Tool by tool. Decision by decision.



Mistake #2 — We Never Cared About Traceability



💬 Be honest — when code changed last sprint, *did anyone* know which tests to re-run, or did you just re-run everything?

♥ What we loved

- ✗ Manual regression planning every release
- ✗ Root-cause-by-memory (ask the senior tester)
- ✗ "Let's just re-run the full suite"
- ✗ Tribal knowledge in people's heads

✗ What we never built

- ✗ Which tests are impacted by this code change?
- ✗ Which requirement does this code implement?
- ✗ What regression risk did my commit just introduce?

AI didn't break our SDLC. It exposed a **debt we refused to pay.**



Act 2 — The Reversal

AI is paying back the debt — but not for us.

The IDE becomes the OS. Code repository becomes project repository. Traceability finally gets solved.

The IDE Is Becoming the OS of Your Project



 Think about Cursor, Copilot, Antigravity. What exactly do they *do* that your old IDE didn't?

Old IDE: A Text Editor

- Syntax highlighting
- Compile / run / debug
- One file in focus at a time
- The project lived elsewhere

New IDE: An Operating System

- ✓ Reads every file in the project
- ✓ Remembers your intent across sessions
- ✓ Talks to Jira, runs browsers, calls APIs
- ✓ Hosts agents that reason across the codebase

The IDE is no longer where you *write code*. It's where the project lives.

From Code Repository → Project Repository

💬 What if every artifact — requirements, code, tests, decisions, bugs — lived under one roof that AI could read?

Code Repository (yesterday)

- Source code only
- Requirements in Jira
- Tests in TestRail
- Decisions in Slack
- Docs in Confluence
- ⚠️ AI sees one slice of your project.

Project Repository (today)

- ✓ Requirements — machine-readable
- ✓ Code — linked to requirements
- ✓ Tests — linked to both
- ✓ Decisions — captured as context
- ✓ Bugs — linked to code and tests
- ✓ AI sees the **whole project**.

Wait — Am I Saying "Replace Jira"?



 I just said "Project Repository." But your team uses Jira and TestRail today. Both things cannot be true. So which is it?

The Project Repository doesn't **replace** Jira. It **adds** a new layer on top.

Jira + TestRail (keep)

Own the **human process layer**. Sprint ceremonies, tickets, approvals, dashboards.

Project Repository (add)

Owens the **machine context layer**. Linked graph of requirements, code, tests, decisions. What AI reads.

Nothing is thrown away. A new layer is built above.

The Artifact: `graph.json`



 How does this file get created? Nobody writes it by hand — AI generates and maintains it.

How it's created

```
# Every AI commit triggers:  
AI-agent scans the diff  
→ linked REQ-ID  
→ updated code_files  
→ linked test_files  
→ fresh coverage numbers
```

Sample entry

```
{  
  "requirement_id": "REQ-142",  
  "title": "Password reset",  
  "code_files": [  
    "src/auth/reset.ts"  
  ],  
  "test_files": [  
    "src/auth/reset.test.ts"  
  ]  
}
```

✓ **Living file.** AI creates it. AI updates it. AI queries it.

Documentation: Finally Getting Solved



 We've begged devs for documentation forever. It never worked. So what changed?

× For Humans

- × Optional — nobody enforced it
- × Cost overhead with no upside for the dev
- × Always outdated within one sprint

For AI

- ✓ Required — AI cannot reason without it
- ✓ Self-generated — written as a byproduct
- ✓ Always current — regenerated on every change

The irony: documentation wasn't solved for *us*. It's being solved because AI needs it more than we do.



Act 3 — Types of AI SDLC

Which type is your team actually operating in?

Different drivers. Different flow diagrams. One uncomfortable truth.

The Types of AI SDLC — With Meaning



💬 Each type isn't a label. It's a different *driver* controlling the SDLC.

1. **AI-Assisted** · Tool-Driven

AI added to existing SDLC.

👉 **Driver = tools**

2. **Spec-Driven** · Requirement-Driven

AI works off structured intent.

👉 **Driver = specifications**

3. **Parallel Stream** · Workflow-Driven

Dev and QA run simultaneously.

👉 **Driver = workflows**

4. **Agentic** · AI-Driven

Agents execute. Humans oversee.

👉 **Driver = AI autonomy**

5. **Regulated** · Governance-Driven

Compliance + audit trail.

👉 **Driver = governance**

Note which one *you're in*. Then which one *you say*.

⇔ Each Type Is a Shift in the Center of Control



💡 These types aren't just "maturity levels." They're different *answers* to: who decides what happens next?

Humans → Tools → Specs → Workflows → AI → Governance

"Each type represents a shift in the center of control of the SDLC."

From humans → tools → specs → workflows → AI → governance. Every step redefines who is really driving.

Type 1 — AI-Assisted SDLC



💬 Does your team just "use Copilot" — but the sprint, ceremonies, handoffs all look identical to 2019?



What's different from 2019?

Individuals use AI tools. Each person ~20-30% faster.

The risk

Sprint structure, handoffs, gates — unchanged. **System output barely improves.**

Type 2 — Spec-Driven SDLC · think BDD, reborn



Remember BDD — Given/When/Then? The idea was right. It just needed AI to make it actually work.

Requirement → AI checks: is it testable? → QA approves the spec → Dev writes code *from spec* (+Copilot) → Tests derive from spec

What's different

AI tools are still used everywhere. But nothing starts until **QA approves the spec**. The spec — not a ticket — is the source of truth.

It's BDD that finally works

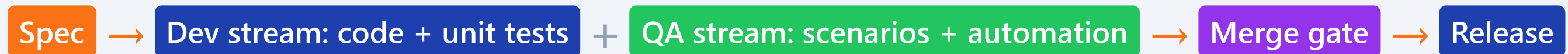
BDD always needed a human to write, maintain, and translate specs into tests. AI does all three. The spec becomes executable.

≡ Type 3 — Parallel Stream SDLC



💬 What if Dev and QA started at the same moment, on the same spec, and synchronized at fixed points?

Same sprint. No waiting. Synchronized at gates.



✓ The win

Same-sprint delivery: built ✓ tested ✓ automated ✓

The risk

Needs strong context sharing. Dev and QA must reason from the *same* spec, not their own version.

Type 4 — Agentic SDLC



💬 What if humans stopped executing and only *governed* — and agents did the work?



The shift

QA becomes an **agent designer and governor**. Not a tester.

The risk

"The agent approved it" is not enough. Human review gates are mandatory — not optional.

Type 5 — Regulated AI SDLC



Banking. Healthcare. Insurance. Fintech. Where "the AI did it" is not a legal defence.

The uncomfortable truth

Most teams are **Type 1** — dressed up as Type 3.
And pretending doesn't help when production breaks.



Act 4 — The Generation Gap

Generation is 2026. Validation is still 2023.

Slow down here. This is the point of the whole talk.

? Why Did Every AI Vendor Chase Generation First?



💬 Think about every AI demo you've seen since 2023. Generation or validation? Now think — why?

🚀 Generation sells itself

- ✓ Writes 200 lines in 3 seconds — *visible*
- ✓ Fits in a 90-second demo video
- ✓ Funds the next round
- ✓ Goes viral on X and LinkedIn

📌 Validation doesn't demo

- ✗ Requires context across a whole project
- ✗ Doesn't fit in a 90-second video
- ✗ Measured by what *didn't* break
- ✗ No viral moment

So generation raced ahead. Validation waited in line.

THE GENERATION GAP

2026 — Generation

2023 — Validation

Three years behind.

That gap has a name. It also has an owner.

This Is QA's Moment



 If validation is 3 years behind generation, who do you think the industry turns to — to close that gap?

The wrong read

"AI will take our jobs." Complain. Get defensive. Wait it out.

The right read

The gap is **territory**. Whoever closes it — owns the next decade of quality.

The generation gap is not a problem. It's a career.



Act 5 — The Tool Landscape

What you actually have to close the gap.

7 categories. Real tools. Real names. Not marketing.

🗪 7 Categories of AI Validation Tools



💬 How many of these does your team use today? How many could you name before this slide?

1. Browser Assistant Agents

Test from the user's eye

2. MCPs (Model Context Protocol)

Give AI keys to the browser

3. AI IDE Browser Agents

Test from inside the dev's world

4. Platform AI

Tie SDLC artifacts together

5. Playwright Agents

Autonomous test lifecycle

6. Cloud-Scale AI Testing

Devices and browsers at scale

7. Runtime Self-Healing

Your framework + AI keys

No single tool does everything. Composition is the skill.

1. Browser Assistant Agents



 What if your browser had an AI that could actually *use* the page — not just read it?

Perplexity Comet

AI browser that navigates, clicks, fills forms — like a real user.

Claude Chrome Extension

Claude inside any website. Read the page. Take actions.

Best for

Exploratory testing, user-journey validation, AI-as-a-user flows.

Watch for

Non-deterministic runs. Complement to regression — not replacement.

🔌 2. MCPs — Model Context Protocol



💬 How do you give *any* AI direct, standardised access to a browser — not just one vendor's AI?

🤖 Playwright MCP

Exposes every Playwright capability over MCP. Any MCP-compatible AI can run a test, query the DOM, navigate.

🔗 Chrome CDP MCP

Chrome DevTools Protocol wrapped in MCP. Direct browser control — DOM, network, console — to any AI.

MCPs = USB-C for AI tools. Plug your Claude / Copilot / Cursor into a browser via one standard.

Why this matters: no vendor lock-in. Your validation logic runs against any AI.

</> 3. AI IDE Browser Agents



💬 What if the same IDE that wrote the code also ran the browser, saw the test fail, and fixed it — all before commit?



GitHub Copilot

Agent mode — browser tools now native.



Google Antigravity

IDE built browser-first. Dev and validation in one loop.



Cursor

MCP-native. Plays any browser MCP directly.

✓ **Why this is a step-change: Dev and QA are no longer in different tools. Validation becomes part of the commit loop.**

4. Platform AI — Tying SDLC Artifacts Together



 Remember mistake #1 — disconnected tools? These are the tools fixing *that*.



Atlassian Rovo

AI across Jira, Confluence, Bitbucket.
Requirements ↔ code ↔ docs in one query.



Claude Plugins

Native connectors: Jira, GitHub, Drive, Slack. Cross-artifact reasoning.



Custom Agents

Your own agents on top of MCP servers — tuned to your SDLC.

This category *operationalises* the Project Repository. The tie-layer across artifacts.

5. Playwright Agents — Autonomous Test Lifecycle



💬 Playwright was already the leader for automation. What happens when it grows agentic capabilities?

Planner Agent → Generator Agent → Healer Agent

What each does

- **Planner:** proposes test scenarios from a spec
- **Generator:** writes executable Playwright tests
- **Healer:** fixes broken locators & timing at runtime

✓ Why this matters

Plan → write → maintain — as composable agents. QA composes. Agents execute. Coverage auditing lives in WF8, not here.

6. Cloud-Scale AI Testing



 AI test generation is great — until you need to run it on 40 device-browser combos. Then what?

BrowserStack AI Testing Agents

Low-code test authoring via AI + execution across real devices & browsers at cloud scale.

AI-assisted flakiness detection

The cloud sees your failures across thousands of runs. AI diagnoses *why* a test is flaky — not just that it is.

✓ **Best for: when your validation has to run at production scale & diversity, not just your laptop.**

7. Runtime Self-Healing Frameworks



💡 Your existing Selenium / Playwright / Cypress framework — can it heal itself? Yes — if you give it an API key.

Test runs → Locator fails → Framework calls AI API → AI proposes new locator → Test resumes

What changes in your framework

Catch locator / assertion failures. Send the DOM + the intent to an AI API. Apply the fix at runtime. Log for review.

Watch for

Self-healing must *log*. Silent auto-fixes hide drift. Every heal is a signal — to QA.



Act 6 — The QA Workflows

11 workflows. Grouped by purpose, not by where they run.

Design · Execution · Intelligence

A QA Workflow Is Not a Test Case



💬 If I ask for "your workflows" — most teams send me a test case template. That's not a workflow. Here's the difference.

A test case

- ✗ Preconditions, steps, expected result
- ✗ One-time artifact
- ✗ Stops when the test ends

A workflow

- ✓ End-to-end loop across the SDLC
- ✓ Continuous — repeats every sprint
- ✓ Learns from production and feeds back

A test case fires once. A workflow runs forever.

The 11 QA Workflows — Grouped by Purpose



DESIGN

Before testing starts

- ✓WF1: Requirements → Scenarios
- ✓WF2: Exploratory Planning
- ✓WF3: Test Data Design
- ✓WF4: Bug Triage Design

EXECUTION

While testing happens

- ✓WF5: Automation Generation
- ✓WF6: Test Maintenance
- ✓WF7: Self-Healing
- ✓WF8: Coverage Audit

INTELLIGENCE

What the system learns

- ✓WF9: Feature Readiness Gate
- ✓WF10: CI Quality Gate
- ✓WF11: Production Feedback Loop

Design shapes quality. Execution delivers it. Intelligence compounds it.

WF1 — Requirements to Test Scenarios

💬 How long does it take your team today, from receiving a user story, to having solid test scenarios the team agrees on?

User Story → AI extracts testable conditions → AI flags ambiguity → QA reviews & approves → Scenarios linked to REQ-ID

✓ The unlock: QA catches ambiguity **before** dev starts. And every scenario is linked back to the requirement — not sitting in a disconnected TestRail folder.

WF5 — Automation Generation



 Your scenarios exist. Your code exists. What's stopping the test code from writing itself?

Scenarios (from WF1) → Code context (from project repo) → AI generates Playwright tests → QA reviews + tunes → Commit

Why it now works

AI has both scenarios *and* code. Not just one. Tests reflect reality — not guesses.

The anti-pattern

Accepting AI-generated tests without reviewing coverage.
More code ≠ more coverage.

Q WF8 — Coverage Audit



💬 When code changed last sprint, how did you know your tests still covered what mattered — or did you just trust the green suite?

Code change detected → AI maps change to existing tests → Gaps identified → QA prioritises & fills → Coverage restored

✗ Gaps this workflow finds that your CI never will:

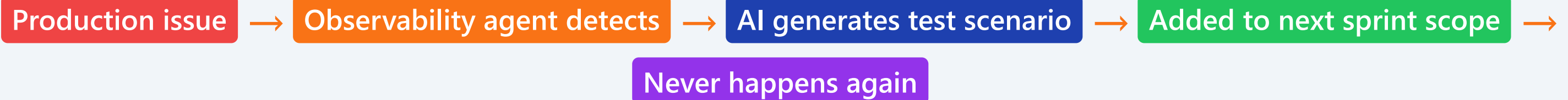
- ✗ New error branch added — no test hits it
- ✗ Payment retry logic changed — old retry test still passes
- ✗ Mobile breakpoint introduced — all tests only run desktop

Green tests ≠ covered tests.

∞ WF11 — Production Feedback Loop



💬 When a bug escapes to production — what actually feeds back into your next sprint? A Jira ticket? Or nothing?



These workflows aren't new. What's new: they can now run at generation speed.

Production stops being the end of the pipeline. It becomes the start of the next sprint.

WF2 — Exploratory Planning



💬 Exploratory testing used to depend on your best tester's instinct. What if AI could amplify that instinct — not replace it?

Feature shipped for testing → AI proposes risk areas from code diff → QA explores guided by AI → Findings logged into project repo

✓ Exploratory becomes **deliberate**, not random. And AI remembers what your team found last sprint — so the next sprint starts smarter.

WF3 & WF4 — Test Data + Bug Triage



💬 Two workflows that die quietly every sprint. How many hours did your team waste last week on data setup and bug deduplication?

WF3: Test Data Design

Scenario → AI generates synthetic data → QA validates edge cases

Realistic test data — on demand, respecting PII, matching scenario intent.

WF4: Bug Triage Design

Bug filed → AI dedupes + prioritises → Assigned to right owner

Bugs reach the right person first time — with linked requirements, code, and similar past bugs.

WF6 — Test Maintenance



💬 Your team's biggest hidden cost isn't writing tests. It's *fixing* tests every sprint. What if that cost dropped to zero?

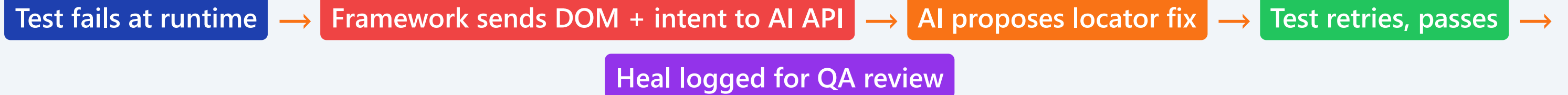


⚠️ **Anti-pattern: AI rewrites tests without QA review. The suite stays green — because it's been gutted. Always keep the review gate.**

WF7 — Self-Healing Tests



💬 A flaky test fails at 2am. Does someone get paged? Or does the test fix itself and log what it did?



✓ The win

Pipelines stay green despite UI drift. No 2am pages for cosmetic changes.

Non-negotiable

Every heal is logged. Silent fixes hide real regressions. Review the heal log weekly.

WF9 & WF10 — The Quality Gates



💬 Today, "ready to release" is an opinion. What if it was a *checked artifact* the system produced automatically?

WF9: Feature Readiness Gate

Feature done → AI checks all WF outputs → Pass / fail + evidence

Did WF1-8 actually run? Did coverage reach target? Did exploratory happen? Pass/fail — not opinion.

WF10: CI Quality Gate

PR opened → AI audits against quality rules →

Merge allowed / blocked

Every merge enforces: tests exist, coverage intact, no silent heals, no broken traceability.

Gates turn *judgement* into evidence. Every release decision becomes auditable.



Act 7 — Tools × Workflows

No single tool completes a workflow. Composition is the skill.

4 stack diagrams. Real tools. Real order.

The Composition Principle



💬 Which vendor sells a complete "QA AI platform"? None. And that's on purpose.

✖ The trap

Buy one AI tool. Expect it to handle requirements, tests, coverage, healing, production feedback. It won't.

✔ The skill

Pick the right tool for each stage of each workflow. Chain them. Own the composition.

QA's new job title: Workflow Composer.

Not tester. Not automator. The person who decides which tools stack together to close a loop.

Stack — WF1 Requirements → Scenarios



- ① **Atlassian Rovo** *reads the user story in Jira, pulls Confluence context*
- ② **Claude Plugin (Jira)** *flags ambiguity, asks clarifying questions to the BA*
- ③ **Cursor / Antigravity** *drafts executable test scenarios, linked to REQ-ID*
- ④ **Playwright MCP** *validates scenarios are testable against current code*
- ⑤ **graph.json** *stores the requirement ↔ scenario link in the project repo*

5 tools. 1 workflow. Composed — not bought.

Stack — WF5 Automation Generation



- 1 **graph.json** *provides scenario + requirement + code file context*
- 2 **Cursor / Copilot / Antigravity** *AI IDE reads scenario + code, drafts Playwright test*
- 3 **Playwright MCP** *dry-runs the test in a real browser, catches immediate issues*
- 4 **Playwright Healer Agent** *adjusts locators / waits until test runs stable*
- 5 **BrowserStack AI** *executes across device / browser matrix at scale*

✓ **From scenario to executable cross-browser test** — in one unbroken chain, at generation speed.

🔍 Stack — WF8 Coverage Audit



- 1 **GitHub / Bitbucket webhook** *triggers on every merged PR*
- 2 **Custom Agent (on MCP)** *reads the diff, queries graph.json for linked tests*
- 3 **Claude / Copilot** *reasons: does the change touch untested code paths?*
- 4 **Jira via Rovo** *raises a "Coverage Gap" ticket with context, linked to the PR*
- 5 **graph.json** *updates the gap register — visible to the whole team*

No new dashboard. No new tool to buy. The audit lives inside the tools you already use.

∞ Stack — WF11 Production Feedback Loop



1 **Observability (Datadog / Sentry / New Relic)** *emits an error event or anomaly*

2 **Custom Agent (MCP)** *pulls the stack trace, correlates to graph.json*

3 **Claude** *proposes a test scenario that would have caught it*

4 **Rovo → Jira** *creates a scoped scenario for the next sprint backlog*

5 **Playwright Agent** *next sprint: generates, heals, runs — automatically*

🔄 **Production bug → regression test — without a human in the loop.** A human *approves*. A human does not *write*.

Stack — WF2 Exploratory Planning



- 1 **Bitbucket / GitHub diff** *what code actually changed since last feature drop*
- 2 **Claude + graph.json** *reasons about risk areas from diff + past exploration logs*
- 3 **Claude Chrome Extension / Perplexity Comet** *QA explores live, AI suggests next steps in real-time*
- 4 **Rovo → Confluence** *findings captured as structured notes, linked to REQ-ID*

✓ **Exploration becomes cumulative.** Each sprint starts with the prior sprint's findings already in context.

Stacks — WF3 Test Data + WF4 Bug Triage



WF3 · Test Data Design

- ① **Scenario from WF1** *defines shape of data needed*
- ② **Cursor / Claude** *generates synthetic data — PII-safe, edge cases included*
- ③ **Custom Agent** *seeds data into test env, rolls back after test*

WF4 · Bug Triage Design

- ① **Rovo / Jira webhook** *triggers on every new bug filed*
- ② **Claude + graph.json** *dedupes, links to code area, suggests owner*
- ③ **Jira** *auto-assigns with priority + linked artifacts*

Stack — WF6 Test Maintenance



1 **Merge webhook (GitHub / Bitbucket)** *fires on every PR merged to main*

2 **graph.json** *resolves "which tests are linked to this code?"*

3 **Cursor / Copilot** *proposes updates to impacted tests, not a rewrite*

4 **Playwright MCP** *dry-runs updated tests to confirm they still validate intent*

5 **QA (human gate)** *approves — or rejects. Never silent auto-merge.*

 **The human gate is non-negotiable. Otherwise the suite stays green because it's been gutted.**

Stack — WF7 Self-Healing Tests



- ❶ **Your framework (Playwright / Selenium / Cypress)** *catches locator or assertion failure at runtime*
- ❷ **AI API key (Claude / OpenAI)** *framework sends DOM + intent + failure to API*
- ❸ **AI proposes fix** *new locator or updated assertion returned*
- ❹ **Framework retries** *test resumes, passes, flow continues*
- ❺ **Heal log (graph.json / dashboard)** *every heal recorded for QA's weekly review*

✓ **No new tool bought.** Your existing framework + an API key + 30 lines of glue code = self-healing.

Stacks — WF9 Feature Readiness + WF10 CI Quality Gate



WF9 · Feature Readiness Gate

- ① **Feature branch + graph.json** *did WF1-8 actually run for this feature?*
- ② **Custom Agent** *checks scenarios, coverage, exploration log, maintenance*
- ③ **Rovo → Jira** *posts pass/fail with evidence attached*

WF10 · CI Quality Gate

- ① **GitHub Actions / Bitbucket Pipelines** *triggers on PR open*
- ② **Custom Agent + graph.json** *audits: tests linked, coverage intact, no silent heals*
- ③ **PR status check** *merge allowed / blocked — with reason*



Act 8 — Integration

QA workflows inside the Dev AI SDLC — not alongside it.

The session title, finally resolved.

⚠️ Current State — Parallel Disconnected Streams



💬 Sound familiar? Does this feel like Mistake #1 — but happening *again*, with AI?

</> Dev AI Stream

- ✓ Cursor / Copilot writing code
- ✓ AI PR reviews
- ✓ Auto-generated docs
- ✓ Merged to main, fast

🔗 QA AI Stream (separate)

- ✓ AI writes test scenarios
- ✓ Separate tool generating Playwright
- ✓ Different dashboards
- ✓ Validation runs *after* merge

⚠️ Two AI streams. Two sets of tools. No shared context. We're re-creating our mistake — at generation speed.

Integrated State — The Unified Loop



QA workflows trigger *inside* the Dev AI SDLC — at every gate.



✓ **One loop, not two streams.** QA workflows are gates on the Dev AI SDLC — not a parallel, disconnected universe.

Match the Workflow to the SDLC Type



💡 Mismatch = failure. Type 4 workflows in a Type 1 team collapse. Type 1 workflows in a Type 3 team leave value on the table.

SDLC Type	Design (WF1-4)	Execution (WF5-8)	Intelligence (WF9-11)
1 — AI-Assisted	WF1 manual	WF5 manual	Skip — not ready
2 — Spec-Driven	WF1 as a gate	WF5 semi-auto	WF10 emerging
3 — Parallel	All 4 active	All 4 active	WF9-11 mandatory
4 — Agentic	Agent-driven	Agent-driven	Fully automated loop
5 — Regulated	All + audit	All + audit	All + human sign-off

Start with your *actual* type. Not the one on your slides.

What This Whole Thing Is Actually For



💬 At the end of every sprint, when leadership asks "can we ship?" — what does your team answer with today?

Today

- ✗ "I think so."
- ✗ "All our tests passed."
- ✗ "QA signed off."

Opinion. Trust. Hope.

With these workflows

- ✓ "Every requirement has linked tests."
- ✓ "Every code change was audited for coverage."
- ✓ "Every production bug last sprint is now a regression test."

Evidence. Not opinion.

Release decisions from **evidence** — not **opinion**.



Act 9 — Where Is This Heading?

AI takes the middle. Humans anchor the ends.

The future of the SDLC — and the AI IDE as the new command center.

↔ AI Has Taken the Middle



💬 Look at any modern SDLC. Where is AI already unstoppable — and where does it still need a human?

🧠 INTENT (human)

- Product definition
- Architecture calls
- Risk decisions

🤖 EXECUTION (AI)

- Code generation
- Testing & integration
- Optimization
- Documentation

👤 ACCOUNTABILITY (human)

- Governance
- Production impact
- Compliance sign-off

"AI has taken the middle. You define the intent and own the outcome. The execution in between is mine."

✂ The Middle Is Expanding. The Ends Are Not.



💬 Today AI executes. Tomorrow AI will propose architecture. So what truly stays with humans — forever?

📄 AI expanding outward

- ✓ Starting to suggest ideas
- ✓ Proposing architectures
- ✓ Making operational calls

The boundary isn't fixed.

⚓ What stays with humans

- ✓ **Ownership** of decisions
- ✓ **Deciding** what matters
- ✓ **Taking responsibility** for consequences

Not tasks — accountability.

The future isn't "AI owns the middle." It's AI owns execution — humans stay anchored at intent and accountability.

The AI IDE Becomes the Command Center



💬 If one environment held *everything* — requirements, code, tests, pipelines — what would your current tool stack look like?

The repo as a living system

Requirements, code, tests, diagrams, release notes, AI configuration, project memory — all version-controlled, all AI-maintained.

Role-aware, intelligent views

PM sees requirements & risk. Dev sees code with context. QA sees coverage & gaps. One system, many lenses.

Jira-PM may be replaced

AI IDE tracks progress from actual code. PM tool becomes redundant.

CI/CD stays — as infrastructure

Build & deploy pipelines run beneath the IDE as the engine.

AI IDE = interface · CI/CD = engine · PM tools = legacy.

The Generation Gap Is Your Job



So here's what I want you to carry home.

AI didn't break QA.

It exposed a debt we refused to pay.

Generation is 2026. Validation is 2023.

That gap is not a problem. It's a career.

If we designed the workflows right — validation runs at generation speed.



YOUR JOB MONDAY MORNING

Explore all 7 categories of AI validation tools.

- 1. Browser Assistant Agents** — Comet, Claude Chrome
- 2. MCPs** — Playwright MCP, Chrome CDP MCP
- 3. AI IDE Browser Agents** — Copilot, Antigravity, Cursor
- 4. Platform AI** — Rovo, Claude Plugins, Custom Agents
- 5. Playwright Agents** — Planner, Generator, Healer
- 6. Cloud-Scale AI Testing** — BrowserStack AI
- 7. Runtime Self-Healing** — your framework + AI API key

Touch each one. Build something small. Compose, don't just consume.

That's how you close the generation gap.

Thank you. · Sudhakar Kakunuri · vibetestq.com

